

Diskless Viewstamped Replication with Unbounded Crash-Stop Resurrection

Simon Massey

Abstract—Unbounded Viewstamped Replication (uVRR) retells David Turner’s construction for keeping a consensus pipeline unbounded through cluster reconfiguration [3] in the vocabulary of Viewstamped Replication Revisited [2]: era-indexed ballots, weighted quorums, the leader casting vote, and unbounded pipelining. The reasoning is Turner’s; what this paper adds is an executable safety ladder — a Lean 4.33.1 development whose every positive result is kernel-checked with runnable evidence — and a working demonstration. The demonstration matters because the failure is measured, not hypothetical: Alagappan et al. documented consensus-based storage losing committed data when crashed nodes rejoined as themselves, trusting state they no longer had [4]. uVRR eliminates that class by design: Crash-Recovery is converted into Crash-Stop, and a crashed node reincarnates as a new node that rejoins as a weight-0 standby (a *standby* is a non-voting member: zero voting weight, part of no quorum, inactive in the Viewstamped Replication algorithm). The checkpoint extends an inherited Lean development of the era-indexed agreement argument with a finite-support weighted-majority theorem, a proposal-era bound from suffix promises, operational acceptor and normal-log invariants, a whole-log selection induction step, and fault-injection controls. We distinguish reproducibility from mathematical correctness and from protocol refinement; counterexamples establish independence of assumptions from claims of universal necessity. The outstanding integration obligations are stated explicitly: protocol refinement, recovery fencing, repeated reconfiguration, client semantics, and progress. The lunet-locks demonstration implements the diskless core for advisory leases and is under active hardening; rejoin-latency, forced-flush, and fault-injection experiments are designed but not yet performed.

Index Terms—Distributed algorithms, replication, reconfiguration, crash recovery, formal verification.

I. INTRODUCTION

VIEWSTAMPED Replication (VSR) was introduced by Oki and Liskov [1]: computations run at a primary, which notifies its backups of what it has done, and when a replica fails the backups reorganize — a *view change* — so that processing continues without user-visible interruption. Viewstamped Replication Revisited (VRR) presents an updated version of the protocol [2]. It works in an asynchronous network like the Internet and handles failures in which nodes fail by crashing. It supports a replicated service running on a number of replica nodes: clients run general operations to observe and modify the service state, so the approach is suitable for implementing replicated services such as a lock manager or a file system. In normal operation the primary orders client operations, replicates each one with a quorum

acknowledgement before committing and executing it, and answers the client. A view change replaces a failed primary: replicas exchange their logs in two rounds, and the new primary installs the log with the greatest last-normal view, breaking ties by length. A crashed replica may rejoin only after a recovery exchange vouches that its state is at least as recent as when it failed. A reconfiguration protocol changes the group membership, and even the number of failures the group can handle, by moving the group between epochs. Notably, the protocol performs no disk writes on the normal path: replicated state, not local storage, provides the persistence.

That diskless design has a sharp edge. Alagappan et al., in a Best Paper at USENIX FAST 2018 [4], show that consensus-based storage can break the invariants of the very protocol the node runs when a node recovers from a crash and rejoins as itself: under the recovery policies the systems actually shipped, a recovering node truncates its log, losing the data locally, and when that node then forms a majority together with nodes that are lagging, committed data is silently lost, violating safety. Their Section 2.3 states exactly this on the printed page 17; their Figure 2 (page 18) walks the failure through the protocol step by step; and their Table 4(a) (page 25) measures it, with the shipped recovery policies recovering correctly in only 46 of 2401 targeted-corruption cases across the systems they tested. Their protocol-aware recovery repairs the class by teaching the storage layer the protocol’s own rules, at the cost of fsyncs on the recovery boundary. Michael et al. had formalized the same boundary for diskless crash recovery [5], [6]; the Appendix B.1 divergence pattern of their extended report is the same class.

This paper takes the position that the amnesia class is better eliminated by design than repaired by a smarter recovery path. The motivating uVRR proposal asks whether a strongly consistent replicated service can replace failed members while continuing to process requests, without forcing its log to disk on the normal agreement path [7]. Unbounded Viewstamped Replication answers by converting Crash-Recovery into Crash-Stop: a low-latency cluster reconfiguration evicts the crashed identity through a forced sequence of unit weight changes, and the node *reincarnates* on restart as a new node that rejoins as a weight-0 standby (a standby has zero voting weight, so it cannot form part of any quorum nor actively participate in the VSR algorithm) and earns its votes back under the leader’s control. Same-identity recovery after volatile-state loss is thereby unrepresentable: the crashed identity is gone, so no recovered-but-amnesiac process is ever trusted again. The network round trips of reincarnation replace the fsync, because “the network is faster than the disk” [8]. The original contribution at stake is diskless strong consistency throughout

VRR view changes, with replicated state, not local storage, providing the persistence.

A statement of attribution comes early. This paper represents David Turner’s Unbounded Paxos leader casting-vote techniques [3], retold here as Unbounded Viewstamped Replication: the construction — era-indexed ballots, weighted quorums, the leader casting vote, unbounded pipelining through reconfiguration — is his, and the reasoning is his, ported from his dynamically reconfigurable consensus clusters to the VRR setting. Crediting a mathematical technique and applying it to a distinct algorithm is valid: Paxos is not VRR. Turner does not use the abbreviation “UPaxos”; this paper does not either. What is new here is the executable safety ladder: every inherited and extended result is replayed as a kernel-checked Lean 4.33.1 development with executable evidence documents, negative controls, and model checks. It is also new here as a working demonstration: lunet-locks, an advisory-lease service running the diskless core, described in Section VIII. The experiments planned for that demonstration — node-kill rejoin latency, the latency delta of a forced synchronous disk flush at the recovery boundary, Maelstrom brute-force fault injection at three and then five nodes, and repetition on cloud deployments — are presented in Section IX as designs, with placeholders in place of measurements and warnings at the section’s top and bottom.

A precise safety answer requires more than an intersection calculation. The membership used by a vote must be agreed; a replacement primary must preserve committed operations; a crashed replica must not cast votes using forgotten state; and a reply must correspond to an operation with the promised client semantics. Progress and elapsed time impose additional assumptions. The central unresolved claim is stated explicitly: *every execution of the intended uVRR message protocol, under a specified crash-recovery model, implements a linearizable replicated state machine across arbitrarily many reconfigurations.* The results below discharge parts of this claim — as kernel-checked component theorems, stated obligations, and a demonstration — as of this checkpoint. They do not discharge it by renaming a ballot a VRR view.

II. SETTING AND TRUST BOUNDARY

Let A , B , and V be node, ballot, and value types. A node set is a predicate $A \rightarrow \text{Prop}$; a quorum family is a predicate on such sets. Define

$$Q \frown R \iff \forall q \in Q, r \in R, \quad q \cap r \neq \emptyset.$$

This relation is symmetric. It is a property of two families, not a requirement that every pair of members within either family intersect. Empty families make some such statements vacuous. Consequently nonemptiness conditions in the agreement theorem must remain visible.

The history model records messages that have ever been emitted. Its predicates include free promises $p(a, b)$, promises carrying a last vote $p(a, b; c)$, proposals, acceptances, and choices. A function $v(b)$ gives the unique value associated with each ballot. The representation presupposes ballot ownership or an equivalent mechanism preventing two values at one

S1	$\text{proposed}(b_1) \wedge \text{chosen}(b_2) \wedge b_2 < b_1 \Rightarrow \mathcal{Q}^I(b_1) \frown \mathcal{Q}^II(b_2)$
S2	$\text{promised}(a, b) \wedge b' < b \Rightarrow \neg \text{accepted}(a, b')$
S3	$\text{promised}(a, b; c) \Rightarrow c < b \wedge \text{accepted}(a, c) \wedge \neg \exists c'. (c < c' < b) \wedge \text{accepted}(a, c')$
S4	$\text{proposed}(b) \Rightarrow \exists q \in \mathcal{Q}^I(b). \forall a \in q. \text{promised}(a, b) \vee \exists c. \text{promised}(a, b; c); \text{ if } P \neq \emptyset \text{ then } v(b) = v(\max P) \text{ for the reported ballots } P$
S5	$\text{accepted}(a, b) \Rightarrow \text{proposed}(b)$
S6	$\text{chosen}(b) \Rightarrow \exists q \in \mathcal{Q}^II(b). \forall a \in q. \text{accepted}(a, b)$

Fig. 1. Single-instance agreement invariants.

ballot; it is not a proof that a wire protocol enforces that mechanism.

The ballot order is well founded and strictly total. The era development uses $B = \mathbb{N} \times \mathbb{N}$ ordered lexicographically, so that unboundedly many rounds are available in each era. This is a mathematical domain; a fixed-width implementation needs a separate overflow argument.

The executable library imports Lean core and other modules in the same development. It has no Mathlib dependency. A proof may use standard Lean axioms `propext`, `Quot.sound`, and `Classical.choice`; these are distinguished from additional protocol hypotheses appearing in theorem types. Printing axioms alone does not expose every protocol assumption: a conditional theorem can have an empty axiom footprint and strong hypotheses. The central generic single-instance theorem has no axiom dependencies. Other results use subsets of the three standard axioms.

III. THE SINGLE-INSTANCE AGREEMENT ARGUMENT

Turner’s construction, following the single-instance argument he recasts from Lamport’s Synod, rests on six history conditions. Fig. 1 shows the invariants preserved by the single-instance agreement argument, redrawn with attribution from Turner’s construction and restated here in our notation; they are encoded as `Synod.Inv`.

The maximal reported ballot in S4 is supplied existentially. An implementation must obtain it from a finite collection and enforce the value rule. The history theorem does not construct a leader’s message handler.

Theorem 1 (Synod agreement). *Assume the six conditions, a well-founded strict total ballot order, and nonempty phase-II quorums. If b and c are chosen, then $v(b) = v(c)$.*

Proof. For a chosen c and a proposed $b > c$, intersect the decision quorum at c with the promise quorum at b . A common acceptor cannot have supplied a free promise at b , since it accepted c . Its reported vote is at least c by condition 3. The greatest reported ballot m therefore satisfies $c \leq m < b$. If $m = c$, condition 4 gives the conclusion. Otherwise apply well-founded induction to the proposal at m , which exists by conditions 3 and 5. Finally, a chosen ballot has a proposal by conditions 5 and 6 and decision-quorum nonemptiness. Trichotomy reduces agreement to the preceding argument or equality of ballots. $\square$

P1	$\forall e. \mathcal{Q}_e^{\text{II}} \frown \mathcal{Q}_e^{\text{I}} \wedge \mathcal{Q}_e^{\text{I}} \frown \mathcal{Q}_{e+1}^{\text{II}}$
P2	a suffix promise $\text{promised}_{\geq j}(a, b)$ implies $e(b) \leq e(\min(j, i_{\max}))$ and excludes acceptances below b at every $i \geq j$
P3	$\text{promised}_i(a, b) \Rightarrow e(b) \leq e(\min(i, i_{\max}))$ and no acceptance below b at i
P4	$\text{promised}_i(a, b; c) \Rightarrow e(b) \leq e(\min(i, i_{\max})), c < b$, accepted $_i(a, c)$, and no acceptance strictly between c and b
P5	$\text{proposed}_i(b) \Rightarrow$ a quorum of $\mathcal{Q}_{e(b)}^{\text{I}}$ supplied free, last-vote, or suffix promises, and the S4 value rule holds for v_i
P6	$\text{accepted}_i(a, b) \Rightarrow \text{proposed}_i(b)$
P7	$\text{chosen}_i(b) \Rightarrow i \leq i_{\max} \wedge e(i) \leq e(b) + 1$ and a quorum of $\mathcal{Q}_{e(i)}^{\text{II}}$ accepted

Fig. 2. Era-indexed agreement invariants.

The checked declaration is `Synod.theorem8`; its intermediate induction lemma is `Synod.lemma7`. This proof follows Turner’s Appendix B.

Turner weakens the classical requirement that all majorities intersect: the phase-I and phase-II quorum families of one ballot need intersect only for the relevant ballot pairs (his section III-B). That weakening is what makes general reconfiguration possible; the inherited development encodes it as S1 rather than as a global intersection assumption.

IV. ERA-INDEXED AGREEMENT AND UNBOUNDED PIPELINING

VRR runs a sequence of agreement instances, one per log position; Turner’s construction indexes their quorum families by *eras*, which play the role of VRR’s epochs. Fig. 2 shows the invariants preserved by the era-indexed agreement argument, redrawn with attribution from Turner’s construction and restated here in our notation.

A. The configuration condition

Let $e(b)$ be a ballot’s era and $e(i)$ the era of instance i . Phase I uses $\mathcal{Q}_{e(b)}^{\text{I}}$ and phase II uses $\mathcal{Q}_{e(i)}^{\text{II}}$. The configuration condition is

$$\forall e, \quad \mathcal{Q}_e^{\text{II}} \frown \mathcal{Q}_e^{\text{I}} \quad \wedge \quad \mathcal{Q}_e^{\text{I}} \frown \mathcal{Q}_{e+1}^{\text{II}}. \quad (1)$$

The reduction also requires proposal and choice era bounds:

$$\text{proposed}_i(b) \Rightarrow e(b) \leq e(i), \quad (2)$$

$$\text{chosen}_i(b) \Rightarrow e(i) \leq e(b) + 1. \quad (3)$$

Ballot order must respect era order. These conditions are separate from Equation (1); the latter alone is not a complete protocol contract.

Lemma 2 (Relevant intersection). *If $b_2 < b_1$, b_1 is proposed at i , and b_2 is chosen at i , then $\mathcal{Q}_{e(b_1)}^{\text{I}} \frown \mathcal{Q}_{e(i)}^{\text{II}}$.*

Proof. The bounds give $e(b_1) \leq e(i) \leq e(b_2) + 1 \leq e(b_1) + 1$. Thus $e(i)$ is either $e(b_1)$ or $e(b_1) + 1$. The first case uses the intra-era half of Equation (1), by symmetry; the second uses its cross-era half. $\square$

Instantiating the single-instance theorem for every instance yields `Paxos.theorem10` (the inherited namespace name is the original development’s). Its type also contains the promise,

value-selection, acceptance, and choice conditions. Agreement holds for every history satisfying them, across any number of eras. This quantification does not establish that an operational sequence of membership changes preserves them.

B. Deriving the proposal-era bound

The inherited `EraLe` hypothesis, Equation (2), bypassed part of Turner’s Figure 2. The extension `MultiPromise.History` restores a known-instance frontier $i_{\max}$ and suffix promises $p_{\geq j}(a, b)$. A suffix certificate expands to a free promise at every $i \geq j$.

Theorem 3 (Suffix-promise reduction). *Suppose instance eras are nondecreasing. A promise at j , including a suffix promise, satisfies $e(b) \leq e(\min(j, i_{\max}))$. Each proposal at i has a nonempty phase-I quorum whose members supply a free promise at i , a last-vote report at i , or a suffix promise starting at some $j \leq i$. Then $\text{proposed}_i(b) \Rightarrow e(b) \leq e(i)$.*

Proof. Choose a member of the proposal quorum. For a point promise, $\min(i, i_{\max}) \leq i$. For a suffix promise, $\min(j, i_{\max}) \leq j \leq i$. In either case monotonicity of instance eras transports the promise’s era bound to $e(i)$. $\square$

`MultiPromise.History.expanded_inv` additionally transports the full promise history to the inherited invariant structure. The resulting agreement theorem retains the known-frontier bound for choices. The expansion is an invariant reduction; the append-only evolution of configuration knowledge and the creation of suffix certificates still need an operational proof. A single certificate can denote every slot $j + d$ for $d \in \mathbb{N}$, as checked by `suffix_unbounded`. This proves representational unboundedness, not completion of those slots.

C. Dynamic configuration changes and the casting vote

In Turner’s construction the configuration sequence and the era sequence are themselves chosen by agreement and held inside the replicated state machine, so no external configuration oracle exists; the sequences may only be appended to, and a node may promise for instances whose era is not yet decided, because the promise is bounded by $e(\min(i, i_{\max}))$ rather than by a known $e(i)$. In normal running the leader holds promises from a phase-I quorum for one large ballot b whose era matches the frontier $e(i_{\max})$, and may then propose for every instance without limit. A reconfiguration appends the replacement configuration as the next era and lets the frontier advance past $e(b)$: proposals on b remain decidable because $e(i) \leq e(b) + 1$, so no pipeline bound is needed. The leader restores normal running by completing phase I at a ballot b' of the new era, sending `prepare(b')` to the new quorum minus itself while the members of the old decision quorum keep accepting on b .

The enabling structure is the *casting vote*: a leader ℓ such that the old decision quorum q and the new promise quorum q' intersect in ℓ alone. The inherited casting-vote lemma (rung 6) assumes q , q' , and a leader ℓ with $q \cap q' = \{\ell\}$. Sending new promises only to $q' \setminus \{\ell\}$ leaves the old acceptance guard

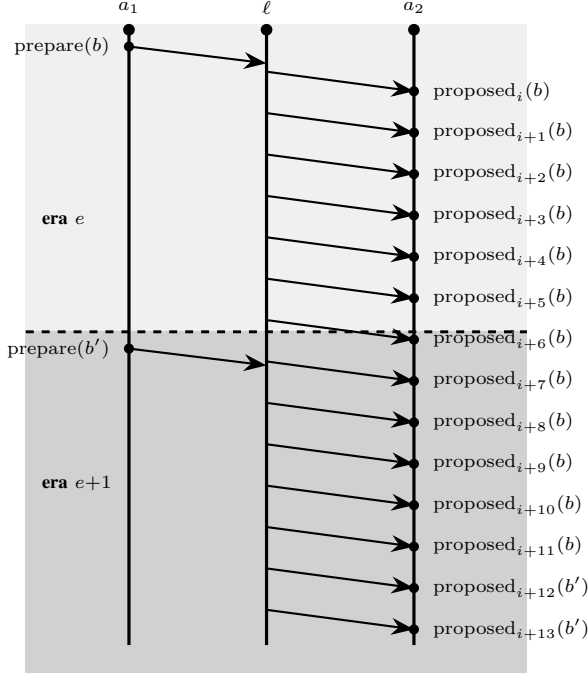


Fig. 3. A fully concurrent reconfiguration.

unaffected at the other members of q . Once ℓ promises, the new quorum is complete — the leader’s self-message has no network delay. This is a schedule-dependent noninterference argument. Availability of those quorums, continued processing by the leader, successful message delivery, and the ability to repeat the procedure require additional proofs. The lemma includes no clock or duration. Rung 23 instantiates the same structure on the reincarnation eras of Section VII-G: in the two-node leader-overlap case — the load-bearing one — the eras’ quorum families meet in the leader alone and every era-boundary quorum pair is decisive at the leader, while the degenerate three-node case’s majorities coincide, so it needs no pivot and exercises no casting vote.

Fig. 3 shows the message flow of a fully concurrent reconfiguration of the Unbounded Viewstamped Replication cluster, redrawn with attribution from Turner’s Figure 3: the leader ℓ continues proposing on ballot b across the era boundary while the new era’s phase-I exchange is outstanding, client requests keep being serviced on the old ballot, and the pipeline never stops. Fig. 4 shows the same reconfiguration in the bounded-pipeline regime that Turner contrasts with his construction — his paper’s term for it was Dynamic Paxos with $\alpha = 2$ (his Figure 4, redrawn with attribution): the pause between instances $i+5$ and $i+6$ is the pipeline bound waiting on the new era’s phase-I exchange. Fig. 5 shows the same reconfiguration in the stop-and-reconfigure regime — his paper’s term for it was Stoppable Paxos (his Figure 5, redrawn with attribution): the reconfiguration command at $i+3$ is proposed before the two preceding instances, and client request processing suspends once the free instances are used up.

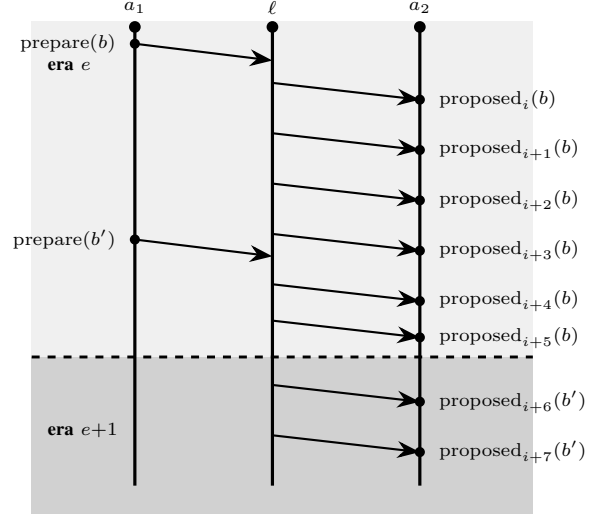


Fig. 4. The bounded-pipeline regime.

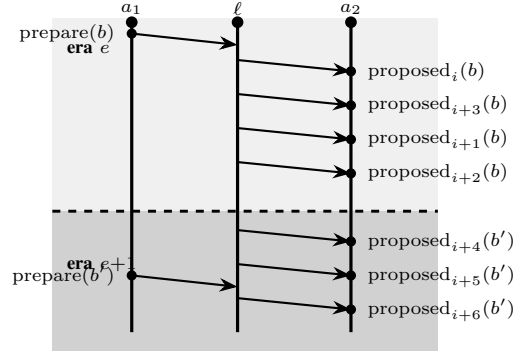


Fig. 5. The stop-and-reconfigure regime.

V. WEIGHTED CONFIGURATIONS

For a finite support list L and natural weights w , let $W = \sum_{a \in L} w(a)$ and $W_q = \sum_{a \in L \cap q} w(a)$. Define $M_L(w) = \{q : 2W_q > W\}$. A duplicate-free list gives the usual finite-set interpretation; repeated entries instead contribute repeated weight. No bound on the cardinality of L is fixed in the theorem.

Lemma 4 (Disjoint selection bound). *For disjoint q, r and natural weights x, y ,*

$$2X_q + 2Y_r \leq X + Y + \sum_{a \in L} |x(a) - y(a)|.$$

Proof. At each node, at most one of the two selections contributes. The required pointwise inequality follows from $2x \leq x + y + |x - y|$ or its symmetric form; if neither selects the node it is immediate. Sum by induction over L . The Lean encoding uses $(x - y) + (y - x)$ with natural truncated subtraction. $\square$

Theorem 5 (Scaled weighted-majority overlap). *For positive integers k, l , if*

$$D = \sum_{a \in L} |kw(a) - lw'(a)| \leq 1,$$

then $M_L(w) \frown M_L(w')$.

Proof. Positive rescaling preserves strict majority. If two majorities were disjoint, their scaled sums would exceed their respective scaled totals by at least one each, since all sums are integers. Their combined excess is at least two, contradicting Lemma 4 and $D \leq 1$. $\square$

This is the weighted-intersection lemma of Turner’s Appendix A, proved here by a pointwise discrepancy bound. `WeightedGeneral.scaled_overlap` is the checked declaration. The unit-change result, equal-rescaling result, and self-intersection follow as corollaries. Zero total weight yields an empty majority family, so self-intersection by itself does not assert availability. The intersection property is the trivial part of the argument: majorities over integer weights that differ by at most one unit per node must overlap, by the pigeonhole principle, and the arithmetic above is all it takes. The real burden is leader overlap across eras — the same value can reach a new era accepted under a different ballot of a different leader, the recovery nuance covered in the author’s Paxos posts [10] — and it is that overlap, not the integer arithmetic, that the surviving proofs must carry.

The uniform bound is sharp. With two nodes, weights $(1, 0)$ and $(0, 1)$ have distance two and admit disjoint singleton majorities. This is checked by `distance_two_counterexample`. Some distance-two changes nevertheless preserve overlap; for example $(2, 2, 2, 0)$ to $(4, 2, 2, 0)$ does. Thus sharpness means that a larger universal allowance would admit counterexamples, not that all changes beyond the allowance are unsafe.

The inherited four-node weighted schedule remains an independent executable example. Its seven rows obey the required overlaps. It is useful for a concrete deployment shape; it is no longer the only evidence for the general weighted construction. Turner’s worked example motivates the same schedule: a naive three-era node swap in a four-node cluster needs all four infrastructure domains independent and leaves the middle era without a casting vote, while the seven-era weight sequence achieves the same swap letting the old and new nodes share infrastructure, with casting votes held throughout. His account of one-at-a-time membership changes under unit weights is the same distance-one rule checked here: under weights in $\{0, 1\}$, consecutive majorities overlap when a single node’s weight changes by one, and the $\text{weight-1} \dots n$ family shows why no larger change is safe. Weightless configurations always satisfy the configuration condition against anything, though an instance decided by an empty family can never be chosen; consecutive instance eras may also skip, which forces a new phase I before any new phase II and recovers the bounded regime of Fig. 5.

VI. OPERATIONAL PROMISES AND FAULT CONTROLS

A. A checked acceptor transition system

An acceptor state is (f, H, R) : a promise watermark $f \in \mathbb{N}$, a newest-first vote history H , and a reply history R . Initially

these are $(0, [], [])$. Acceptance requires $f \leq b$; a promise requires $f < b$. The transitions are:

$$\begin{aligned} \text{accept}_b(f, H, R) &= (b, b :: H, R), \\ \text{promise}_b(f, H, R) &= (b, H, (b, \text{head? } H) :: R). \end{aligned}$$

An empty head produces a free reply; otherwise the reply carries the latest accepted ballot. The histories are ghost evidence for the proof. This module does not prescribe retaining an unbounded physical list at a replica.

Theorem 6 (Historical promise safety). *In every state reachable by a finite execution, a free reply at b excludes votes below b . A reply carrying c at b establishes $c < b$, an acceptance of c , and absence of acceptances strictly between c and b .*

Proof. Strengthen the induction invariant: all votes are at most the watermark; the head vote dominates its tail; every emitted reply’s ballot is at most the watermark; and each reply satisfies the stated historical conditions. An acceptance preserves every old reply because its ballot is at least the old watermark. A promise reads the maximal vote and raises the watermark above it. Neither transition removes prior evidence. Induction on reachable states proves the claim. $\square$

The declarations are `Acceptor.reachable_inv`, `free_forbids`, and `report_last`. They discharge S2 and S3 for a single-era acceptor. They do not establish the proposal value rule, an entire log’s view-change selection, or restoration after volatile-state loss. Extending the component to era-tagged views also remains an explicit integration task.

B. What the negative controls establish

The controls separate two questions: whether the proof notices an altered definition, and whether removing an assumption admits an inconsistent history. The first is measured by compiler rejection; the second by constructing and checking a counterexample with the remaining conditions intact.

- *Cross-era overlap.* The inherited two-node history satisfies intra-era overlap and the other encoded era-history conditions, but chooses both Boolean values at one slot when the cross overlap is absent.
- *Value selection.* One acceptor reports ballot 0 at ballot 1, yet the two proposals use different values. All listed single-instance conditions except S4 hold and both ballots are chosen.
- *Promise fencing.* One acceptor supplies free promises at two rounds despite its earlier vote. All listed conditions except S2 hold, and agreement fails.
- *Nonempty decisions.* Empty phase-II quorums permit choices with no proposal or acceptance. The six history invariants hold vacuously where appropriate, and the two selected values differ.
- *Operational guard.* After a free promise at round 2, accepting round 1 without checking the watermark violates the reachable invariant.

These examples establish independence within the specified contracts. They do not prove a universal necessity theorem

over all consensus algorithms, nor do they enumerate every combination of omitted assumptions.

Separately, the mutation harness replaces the checker’s universal quantifier over quorums by an existential one, and replaces a safe concrete quorum family by an unsafe family. Lean rejects each altered source while accepting the unchanged control. An injected `sorry` demonstrates a different failure mode: compilation succeeds but `#print axioms` exposes `sorryAx`. The harness bounds child memory, work and wall time and modifies only temporary copies. Infrastructure failures are not counted as killed mutants.

Table I maps Turner’s lemmas and figures onto the checked runs; rung numbers are the ladder’s own. Every entry states what is kernel-checked as of this commit; no entry asserts an end-to-end protocol theorem.

VII. THE BOUNDARY BETWEEN UNBOUNDED VIEWSTAMPED REPLICATION AND VRR-2012

VRR-2012 is a replication protocol, with client handling and recovery in addition to the agreement mechanism [2]. Its normal path requires ordered log preparation and a quorum of acknowledgements before execution. View change selects the log with the greatest last-normal view, breaking ties by length. That whole-log selection rule needs a proof relating logs from different views; a per-slot maximal-vote rule is not automatically the same operation.

The distinction becomes essential after crashes. A recovering replica is excluded from normal processing and view changes. Its recovery exchange requires responses from distinct peers, a fresh nonce, and state from the primary of the greatest reported view. The preceding round of `StartViewChange` messages provides replicated knowledge of view advancement. The paper’s Section 8.2 gives a failure scenario when that round is removed. Replacing volatile state by an acceptor watermark that never disappears would bypass precisely this obligation.

The 2012 reconfiguration protocol stops taking new client requests in the old epoch, transfers the committed state, and activates the replacement group. Section 7.5 explicitly discusses the resulting delay and prewarming as a way to reduce it. The `uVRR` objective instead calls for progress during the transition. Equation (1) and casting-vote scheduling address part of that difference, while the interaction with recovery remains to be proved.

A. Checked whole-log selection

The executable `ViewSelection.select` takes a seed report and a list of additional reports, enforcing nonemptiness without a fallback value. It selects by last-normal view and then log length. Each report’s committed frontier is bounded by its length in the report type. The checked results establish that the output belongs to the input collection and has maximal rank. For reports of length at most M , `scalar_rank_equivalent` proves that the model’s scalar rank $v(M + 1) + n$ has exactly the same ordering. A counterexample shows that omitting the length bound can misrank reports.

The prefix-preservation result is the induction step for a single era. Let a prefix P have committed in view v . Intersect its commit quorum with the reporting quorum and obtain a report from a common sender. The explicit voter-history premise says that this report has retained view at least v and, at equality, contains P . The induction hypothesis supplies preservation for reports from strictly later normal views. If the selected report is later than v , that hypothesis applies. Otherwise the witness and selected report are both from v . Same-view log comparability and maximal selected length then imply that the witness log, and hence P , is a prefix of the selected log. `ViewSelection.quorum_preserves` checks this argument.

The selector itself only compares ranks. The surviving-voter and log-origin premises still require proofs from message and recovery transitions. A two-report fault control admits divergent uncommitted suffixes in different views: an older two-entry log loses to a newer one-entry log. A length-only selector chooses the older log and loses the newer committed entry. These results close the selection subproblem while preserving the distinction between a checked induction step and a complete reachable-state proof.

B. Normal-log message provenance

`NormalLog` first checks the fixed-view message argument. The primary starts from an arbitrary base and appends. `StartView` installs that base once; normal replicas accept authentic prepares at their next slot and emit reports copying their logs. Persistent messages permit delayed, reordered, and duplicate delivery attempts. The invariant makes replica logs, sent prepare prefixes, and reports prefixes of the primary history. At receipt, the replica and pre-send logs are equal-length prefixes of that history, hence equal; appending the sent entry preserves the invariant. The pre-send log is ghost evidence, not a content check in the handler. `reports_comparable` follows from prefix comparability, and `replica_history` proves retention over finite continuations. Bypassing next-slot checking admits an authentic second entry at slot one. This component supplies the local argument integrated below; its arbitrary initial base alone does not justify cross-view safety.

C. Shared multi-view preservation

The next integration is now checked. `ViewFence` models a local replica installing higher normal views, appending in normal mode, raising its floor before reporting, and retaining immutable vote and reply histories. Its `report_covers_vote` theorem bounds every historical vote below a report’s target by the reported last-normal view; at equality, the voted log is a prefix of the report. This remains true of votes later in the trace. The selector therefore consumes a derived voter-history property. Strong induction discharges the strictly-later-view premise over activated views; a concrete two-view execution demonstrates that the interface is inhabited.

`LogProvenance` joins the local histories to a shared message model. Each view has one immutable installation base and an append-only primary history. Activation requires

TABLE I
TURNER’S RESULTS AGAINST THE CHECKED LADDER.

Turner item	Content	Checked evidence
Fig. 1 invariants S1–S6 Theorem 8 (via Lemma 7)	single-instance history contract agreement conditional on S1–S6, order, nonempty decision quorums	Synod.Inv, rung 3 Synod.theorem8, Synod.lemma7, rung 3, axiom-free
Fig. 2 invariants P1–P7 Lemma 9 Theorem 10 P2–P5 promise machinery	era-indexed history contract relevant intersection across eras per-instance agreement across eras proposal-era bound from suffix promises and the known frontier	Eras.P1–P7, rung 4 Paxos.lemma9, rung 4 Paxos.theorem10, rung 4 MultiPromise.History, rung 11; cross-era witness, rung 5
Appendix A weighted lemma	scaled weighted-majority overlap at distance one	WeightedGeneral.scaled_overlap and corollaries, rung 9; concrete schedule, rung 7
Section V examples	unit-weight one-at-a-time rule; correlated-failure weight schedules	distance-two sharpness witness, rung 9; four-node schedule, rung 7
Section IV-D casting vote	guard noninterference and phase-I completion at the leader	CastingVote, rung 6; reincarnation-era instances, rung 23

a quorum of authentic reports and uses the executable latest-normal-view/length selector. An issued prepare carries a view, next slot, and entry; its pre-send log is ghost evidence. Installation reads an activated base. Receipt checks normal mode, view equality, and the next slot. Fencing and report emission update the local replica state. There is no guard testing global committed-prefix agreement.

The invariant establishes that each replica log, historical vote, and report extends its retained view’s base and is a prefix of that view’s primary history. Each activated base retains its selection certificate because report histories persist. These facts discharge both same-view comparability and base-extension provenance in the view induction. `later_base_preserves` proves preservation by every later activated base. `committed_comparable` proves that any two quorum-voted prefixes in a reachable state are comparable; `committed_equal` specializes this to equal-length prefix equality. The model bounds neither log length, view numbers, nor activation count, and fixes one self-overlapping quorum family. A concrete trace commits a nonempty prefix in two successive views. A delayed old-view prepare has the expected next slot after installation; bypassing view equality permits an unsupported new-view vote. The constructive control and source mutation both check this boundary.

This is a complete safety induction for the stated crash-free shared model, not yet the requested uVRR theorem. Unique source activation is enforced by an unused-view guard, and source histories may continue issuing prepares after replicas have fenced that view. Their eventual acceptance is constrained by replica guards. A refinement must justify these abstractions from unique primary activation and the concrete wire protocol. Crashes, first-round view-change knowledge, arbitrary membership changes, client responses, and progress remain outside this model. Ghost histories do not imply durable storage at replicas, but they do not prove recovery after volatile-state loss.

D. Recovery fencing: a checked boundary

`RecoveryFence` proves a narrower persistence result. Suppose a fixed support quorum is simultaneously online with bounds at least b , and historical responses carry causal recipient-episode identifiers. Crashing erases a local bound and advances the environment’s fresh episode identifier; recovering nodes cannot answer. Recovery uses distinct current-episode responders and chooses a maximal bound. Induction proves that every online member of the support retains a bound at least b through arbitrary finite crash/recovery continuations. The quorum checkpoint is a premise, not a consequence of collecting asynchronous view-change messages. The model does not restore logs or establish progress. A stale-episode witness and a source mutation check the freshness boundary.

Classic diskless crash-recovery designs are exposed to a known failure pattern: Michael et al. describe delayed view-change and recovery divergence in Appendix B.1 and Figure 1 of their extended report [5], [6]; a replica forgets that it sent a view-change message and recovers into a lower view, and the delayed message later lands under a new leader that overwrites committed decisions. That is literature about the classic crash-recover class, not an open bug here: uVRR eliminates the class by construction. A crashed node whose durable superblocks record an unflushed session reopens under a new incarnation (Crash-Stop-Self-Evict), the leader evicts the old identity, and the new identity rejoins as a weight-0 standby that may receive streams but whose messages are ignored. Same-identity recovery after volatile-state loss is therefore unrepresentable, and the stale-episode rule above is precisely the ingress rule the reincarnated standby obeys. Their crash-vector construction provides the collection primitive the reincarnation leader needs.

E. Crash-vector collection

Following Algorithm 1 and Definition 6 of the extended report [6], let a reply r contain sender s_r , vector $v_r : A \rightarrow \mathbb{N}$,

and a payload. The collector stores a known vector k and replies R . Upon receiving r , it computes

$$k'(a) = \max(k(a), v_r(a)), \\ R' = \{q \in R :: R \mid k'(s_q) \leq v_q(s_q)\}.$$

The implementation uses a list; quorum membership is the set of sender identities. Thus duplicates do not create additional identities. Induction proves every retained vector is pointwise bounded by k . The filter gives the converse inequality at each retained reply's own sender. Consequently,

$$\forall r, q \in R, \quad v_r(s_q) \leq v_q(s_q),$$

the published crash-consistency condition. A known-stale reply remains ineligible as the frontier grows. A concrete three-node example shows that an unfiltered two-sender quorum violates this condition; filtering removes the stale reply and a current replacement restores collector quorum size. Removing the filter in a temporary source copy causes Lean rejection.

This is a collector theorem, not a recovery theorem. The published protocol requires recovery itself to acquire and propagate its new incarnation through a crash-consistent quorum; in uVRR that acquisition is the reincarnation path the four-superblock identity contract drives. Its quorum-knowledge persistence argument also covers reconstruction, message freshness and resending after pruning. Those obligations are not discharged by attaching vectors to ordinary VRR replies. The report's termination condition requires an appropriate quorum stable over the acquisition interval; arbitrary churn with a bound on simultaneous recoveries alone is not a termination theorem. Mechanizing that temporal argument and composing it with log safety remain open.

F. Acquisition ordering and operational certificates

AcquisitionOrder checks the secondary induction of Theorem 3 in the extended report [6]. With asynchronous per-participant witness times and a generic stable property, an on-line participant either retains its witness knowledge or inherits it from an earlier response quorum. Quorum intersection and forward ordering of those responses relative to the witness events discharge a strong induction over time, using core Lean's `Nat.strongRecOn`. A separate lemma derives that forward ordering from crash-vector consistency, conditional on recovery participants retaining incarnation knowledge. A countermodel satisfies every other origin condition except forward ordering and loses the property; a source mutation that removes the ordering premise is rejected.

RecoveryAcquire then makes the acquisition operational, following Algorithm 1. Crash, start, emit, answer, collect and finish transitions act on an immutable authenticated message history. A crash erases all protocol knowledge and begins a fresh logical incarnation; request identity is an explicit echoed (incarnation, sequence) pair in addition to the response vector check. Induction over reachable states proves that every finished certificate is a crash-consistent quorum for exactly its request and incarnation, and that every historical reply originated at an operational sender. A three-node trace completes recovery at incarnation 1. An authentic reply from the

same incarnation but an older request violates freshness when replayed into a new acquisition, which separates sequence freshness from incarnation freshness. Mutations that answer from a recovering sender or accept a stale request are rejected.

These are acquisition mechanics with one active acquisition per node. Quorum-knowledge persistence across the acquisition interval, value reconstruction, the read quorum, mapping the echoed (incarnation, sequence) identity to the superblock incarnation contract, log integration and liveness remain open.

G. Unbounded Viewstamped Replication reincarnation

uVRR does not implement crash-recovery. It enforces Crash-Stop-Self-Evict, called *reincarnation*: a node whose volatile state was lost reopens as a different node. Durable state is four TigerBeetle-style superblocks. A restart that reads all four as `flushed` marks them `unflushed` and continues normally; a restart that reads any `unflushed` block is dirty, bumps its incarnation, and rewrites all four blocks as the new identity marked `flushed`. Any read that observes a higher identity adopts it. Once the eviction is initiated it must continue. A *reincarnation* message carries the identity pair (old, new) to the leader; the durable incarnation supersedes the fence machinery's ghost generation as the freshness carrier where the two meet, while the fence and acquisition theorems above remain in force with the carrier re-instantiated.

The leader stays live for client requests and drives a forced weight sequence from the voting-weights rules [9]: the exiting node's weight goes $1 \rightarrow 0$ (a Leave), the new identity joins at weight 0 (a Join), and the new identity goes $0 \rightarrow 1$. Each step is a unit-weight change, so the strict majorities of consecutive configurations overlap by Theorem 5's corollaries and every intermediate era is quorum-safe on its own; a leader dying mid-sequence leaves only safe eras that a new leader can continue. Table II instantiates the sequence at unit weight for the three-voter cluster A, B, C with B the leader and C the crashed identity that returns as D : the start configuration is $(1, 1, 1, \infty)$; era 1 drives the leader's batch `DECREMENT(C) + JOIN(D)`, reaching $(1, 1, 0, 0)$; era 2 drives `INCREMENT(D) + LEAVE(C)`, reaching $(1, 1, \infty, 1)$. In the table a $-$ is not a weight: it marks an identity that is not a member of the configuration in that era, whereas 0 is the weight of a standby member that the cluster still holds in its order — which the reincarnated node is between the two eras while it is streamed preemptively. The old identity C is evicted by Crash-Stop-Self-Evict, never a recovered amnesiac. If the sequence runs in the doubled scale of a weight-doubling plan, the return halve is integral only when all weights are even, so the corner sequence inserts one extra unit increment on the new identity before the halve. The leader streams the first reconfiguration, then client traffic, and preemptively streams to the reincarnated node as a weight-0 standby; messages to it are fine and messages from it are discarded by the standard membership check, which is the same rule the crash-vector collector's stale-reply filter realizes. If the leader crashes, the cluster must reach a stable leader before the reincarnated node forces its old identity's eviction.

Fig. 6 shows this forced sequence as a space-time chart drawn in the grammar of Turner's message diagrams [3]:

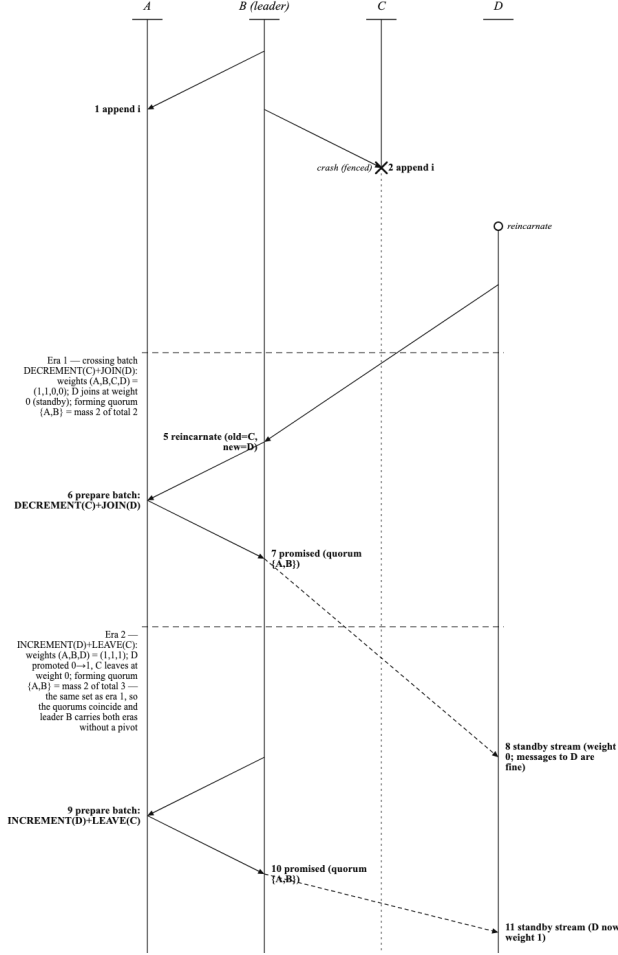


Fig. 6. The reincarnation forced sequence.

TABLE II
REINCARNATION FORCED WEIGHT SEQUENCE.

Era	A	B	C	D
start	1	1	1	—
era 1: DECREMENT (C) + JOIN (D)	1	1	0	0
era 2: INCREMENT (D) + LEAVE (C)	1	1	—	1

four lifelines *A*, *B* (the leader), *C*, and *D*, time flowing down; a numbered diagonal vector per step; the bold cross where *C* crashes and is fenced, with its dead segment dashed below; the open circle where *D*’s lifeline begins; the dashed era-1 separator carrying $\text{DECREMENT}(C) + \text{JOIN}(D)$, with *D* joining at weight 0 as a standby and receiving the dashed preemptive stream; the era-2 separator carrying $\text{INCREMENT}(D) + \text{LEAVE}(C)$, with *D* promoted to weight 1 and *C* leaving at weight 0; and the forming quorum $\{A, B\}$ promised in both eras, so no casting-vote pivot is exercised on this path. The chart is rendered from the tracked source diagrams/reincarnation.svg.

The economics defer disk flushes: nano-state fits in memory, so disk flushes can be deferred, and the network round trips of reincarnation are cheaper than fsyncs because “the network

is faster than the disk” [8]; the four-superblock mark is one fsync amortized over the whole epoch. Same-identity recovery after volatile-state loss is thereby unrepresentable rather than repaired: classic VRR-2012 diskless quorum recovery is not performed, so the class that design exposes — Michael et al.’s Appendix B.1 divergence pattern [6] and the truncate data-loss finding of Alagappan et al.’s Section 2.3 [4] — is avoided by construction, and a zero-weight standby may speculatively recover its log non-votingly as future work. The formalization of the superblock contract and the forced sequence against the fence and acquisition machinery is a concrete proof obligation, recorded with the reincarnation rung’s stated obligations.

H. The design model and VRR-2012 coverage

The repository’s `VrrCoreEras.tla` is explicitly a design model for a single era transition, rather than a statement that the Rust implementation already refines it. Its gate checks both cross-era view/commit directions and fence intersections. The Lean history theorem’s narrower directional overlap must not be used to remove those checks without proving the relevant transition refinement. Table III records the outstanding source-level obligations, including optional VRR optimizations whose semantics differ from the base protocol; “Open” means no end-to-end Lean result is claimed for that obligation.

VIII. THE DEMONSTRATION: LUNET-LOCKS

The demonstration is `lunet-locks`, a runnable, fixed-membership advisory-lock service. It orders lock requests with `vrr-core`, the sans-IO Viewstamped Replication Revisited core that accompanies this formalization: one total function maps a tick, an input message, and a state to a new state and the messages to send, and the host owns time, transport, storage, packetization, threading, and naming. Clients speak newline-delimited JSON over TCP; replicas exchange the core’s binary datagrams over raw UDP; the service process is LuaJIT and libuv (the Lunet runtime, version-pinned and hash-verified), and a small Rust adapter drives the core and owns the lock state machine.

A. Components

The adapter instantiates the concrete core replica over a segmented log with weighted-majority quorums, running volatile stability: protocol state lives in quorum memory, an empty journal is provisioned at boot, and the only bytes the service ever fsyncs locally are the recovery nonce file. The adapter retains the tick clock (a monotonic value clamped against wall-clock regression), exactly-once reply correlation keyed by the client’s message identifier, and guard checks that re-validate every replicated operation entry — in Prepare, DoViewChange, StartView, RecoveryResponse, and NewState messages — against the lock service before the entry reaches the core. The Teal/Lunet wrapper owns sockets, TCP framing, peer source validation, and forwarding; a UDP peer envelope carrying a membership fingerprint guards the fixed membership at the transport boundary, so differently configured clusters cannot merge. An append-only lock-event journal, a

TABLE III
VRR-2012 COVERAGE AT THIS CHECKPOINT.

Source section	Required obligation	Checked evidence and present boundary
1–3: model and architecture	Crash-only failures, authenticated identities, network model, deterministic execution, replica thresholds	Explicit assumptions; quorum intersection is checked. No general environmental refinement theorem.
4.1: normal operation	One value per view/slot, ordered preparation, distinct quorum support, commit before execution, request deduplication	Conditional slot agreement; operational acceptor promise rules and fixed-view log provenance. Proposer, clients, and execution integration open.
4.2: view changes	Sender status, two message rounds, latest-normal-view/length selection, preservation of every committed prefix	Shared crash-free model proves whole-log compatibility across unbounded views. Concrete protocol refinement and the reincarnation proof (the Crash-Stop-Self-Evict forced weight sequence) remain open.
4.3: replica recovery	Exclusion until fresh quorum evidence and current-primary state; restored votes consistent with prior activity	Conditional fence retention and crash-vector collection checked; the classic crash-recover class eliminated by construction (Crash-Stop-Self-Evict reincarnation).
4.4–4.5: applications and clients	Chosen nondeterministic inputs; monotone request identifiers across client crashes	Explicit application/client refinement obligations, open.
5.1–5.3: practical transfer	Checkpoint fidelity, suffix completeness, garbage collection, view-sensitive truncation, short view-change messages	Open; source paper’s full-log argument does not validate optimized transfer automatically.
6.1–6.2: witnesses and batching	Witness participation roles; batch semantics and operation order	Optional refinements open. No correctness claim inherited merely from batching.
6.3: fast reads	Lease exclusion and clock assumptions, or the weaker semantics of backup reads	Open; the backup-read variant is not claimed linearizable by the source.
7.1–7.5: reconfiguration	Epoch establishment, transfer completion, safe retirement, duplicate messages, client routing	Era-history reduction and quorum calculations checked; operational repeated uVRR reconfiguration open.
8.1–8.3: correctness arguments	Compose normal operation, recovery and reconfiguration; state progress assumptions	Component safety results checked. Composition and liveness open.
9: conclusion	Relate the completed results to the replication service	No additional theorem is inferred from the source’s summary.

REST/WebSocket feed, and a console are observability only: the journal never participates in replication or recovery, and a journal error disables journaling without affecting the service path.

B. The rejoin path implemented

Every boot enters fenced recovery stamped with a fresh durable nonce. The nonce file is updated atomically — write, fsync, rename, parent-directory sync — and the new value is used as the recovery input’s tick, so per-node monotonicity holds across the two clock sources. Recovery needs a live quorum: a restarted node rejoins from quorum memory, installing the latest fenced view’s history and re-applying its committed suffix. A rolling single-node restart therefore recovers from the surviving members, while a simultaneous full-cluster loss forfeits whatever the quorum held, and an operator re-bootstrapping a lost cluster must do so only after all outstanding leases can no longer be valid.

Membership is fixed for one process lifetime, so the reincarnation eviction of Section VII-G — the forced weight sequence that replaces the crashed identity with a new one — is not exercised by this service. What the demonstration implements is the classic recovery boundary whose amnesia exposure motivates reincarnation, hardened with durable-

nonce fencing: the fresh nonce is exactly the freshness ingress that the reincarnation design’s stale-episode rule realizes, and a restarting node can never complete recovery with a stale identity stamp. Converting the eviction sequence into this service is the next integration step.

C. The lock and lease API

The client protocol has three operations, one JSON object per line over TCP. Every request carries a message identifier, a stable client identifier, an increasing request number, and a lock identifier; retries reuse the exact envelope, and replication deduplicates by (client, request number), replaying the prior reply. `get` returns the live lease on a lock, or null. `set` grants or renews a lease — a lease identifier, a holder identifier, and an expiry — and a lease is live only when its expiry is later than the execution time; an expired candidate is rejected, an absent or expired incumbent is free, and a live incumbent may be renewed or replaced only by the same holder. `release` removes a live lease only when both holder and lease identifier match, and is idempotent for a missing or expired lease. Lease expiry is what makes the diskless design safe to operate: a crashed holder’s locks free themselves when the lease lapses, without any recovery path trusting the holder’s lost state.

D. Test and hardening status

The service is exercised from several directions: a pure-Teal test suite and an FFI-boundary test suite for the lock service and cluster configuration; a Cerulean format gate and Cyan type checks; the Rust adapter’s checks, tests, and release build, which depend on the exact pinned `vrr-core` git revision fetched over HTTPS; a three-process smoke test covering acquire, read, contention, release, reacquisition, expiry takeover, and a one-replica restart with a live quorum; and a thirty-second, three-datacenter lease-failover simulation that kills the observed lease holder every three seconds and verifies takeover within five seconds, exiting nonzero on a conflicting holder. The same simulation runs against a stable Docker cluster. CI runs on Linux for regular changes and a full platform matrix with packaged releases for tagged versions.

It is a working implementation under active hardening and testing, not a finished demonstration of the reincarnation protocol.

IX. EXPERIMENTAL RESULTS: DESIGNS, NOT MEASUREMENTS

WARNING: THIS WORK HAS NOT BEEN DONE. The experiments below are designs; all numbers and charts are placeholders (marked “dummy”).

The claim under test is diskless low latency: rejoin through reincarnation is faster than paying a forced disk flush at the recovery boundary. The decision rule for the campaign: if the measured rejoin latency of E1 is slower than the forced-flush baseline of E2 at the same sample count, the diskless claim collapses — if recovery is slower than doing disk flushes, nothing has been achieved. The system under test for all four designs is the `lunet-locks` demonstration of Section VIII; every number below is the placeholder `xxxx`, to be filled at run time.

A. E1: node-kill rejoin latency

The design kills one voting node that is not the leader, rotated across all non-leader voting nodes of the three-datacenter `lunet-locks` cluster, for $k = \text{xxxx}$ kill/rejoin cycles, with continuous competing lock traffic at the lease cadence throughout. Each iteration begins only after the previous rejoin has completed serving and a soak interval has elapsed, so no measurement is contaminated by the previous one. For each iteration the harness waits for steady state, kills the node’s process — Crash-Stop: volatile state lost, the node is by construction a different node on reopen — and the iteration ends when the reincarnated node has rejoined as a weight-0 standby, been walked back to voting weight by the leader’s forced sequence, and is again a voting, serving replica. The metric is the kill-to-rejoin-serving latency, one sample per iteration: p50 `xxxx` ms, p90 `xxxx` ms, p99 `xxxx` ms, max `xxxx` ms, mean `xxxx` ms, and the count of eras and view changes consumed per rejoin, `xxxx`. Fig. 7 shows the designed chart: kill iteration on the horizontal axis, rejoin-serving latency on the vertical axis, one series per milestone of the forced sequence (standby rejoin at weight 0, voting again at weight 1). No data points exist yet; the chart is watermarked “dummy”.

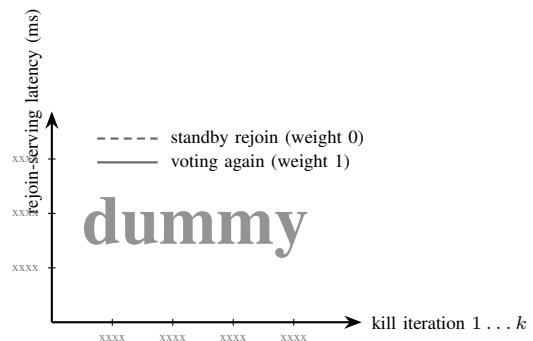


Fig. 7. E1: node-kill rejoin latency.

B. E2: disk modes at the recovery boundary

What does the diskless default save, in latency, against paying a forced flush at the recovery boundary? The design builds custom `lunet-locks` binaries, identical in every respect other than the durability behavior at the recovery boundary — the point at which a reincarnating node durably records state before rejoining serving:

- *Diskless (the default)*. No durable writes at the recovery boundary; quorum memory is the only protocol state, and the recovery nonce file remains as shipped.
- *Naive single write (the latency baseline)*. One 4 KiB block write followed by `fsync` at the recovery boundary. Per the FAST’18 finding [4], even a single write plus `fsync` is not fault-safe against real disk faults; the variant is the latency baseline only, never a safety claim.
- *Double-ring write*. A TigerBeetle-style double write at the recovery boundary: one 4 KiB block of data plus a checksum header to the first WAL ring, then the header and checksum with no payload to the second WAL ring, the two rings spaced apart in separate 4 KiB-sector-aligned zones so the two checksum copies sit in different erasure blocks. Both flush variants carry fake data only, with no read-back: they measure write-and-flush latency and never reconstruct state from what was written.

Each variant runs E1’s procedure exactly, $k = \text{xxxx}$ iterations per variant, cold counts. The metrics are the kill-to-rejoin-serving distribution per variant (percentiles as in E1) and, for the flush variants, the recovery-boundary flush latency itself per iteration. The planned report is the rejoin-latency delta: diskless median `xxxx` ms, naive-flush median `xxxx` ms, double-ring median `xxxx` ms, with tail figures `xxxx` ms at the ninth decile. The delta is the utility test: if reincarnation is slower than the forced flush, the diskless design has achieved nothing. Fig. 8 shows the designed chart: node restart event on the horizontal axis, rejoin-serving latency on the vertical axis, one series per build (diskless default, naive single write, double-ring write). No data points exist yet; the chart is watermarked “dummy”.

C. E3: Maelstrom brute force at three and five nodes

Does reincarnation hold consistency under brute-force fault injection? Jepsen’s Maelstrom drives the cluster under its fault

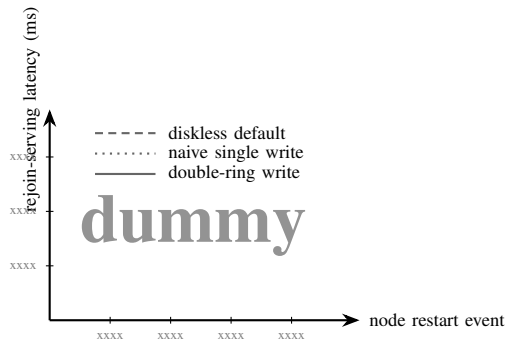


Fig. 8. E2: disk modes at the recovery boundary.

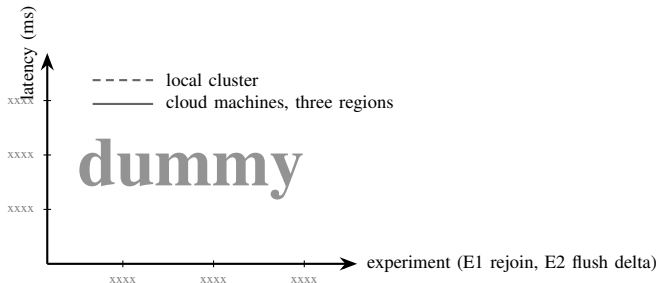


Fig. 9. E4: cloud repetition of E1 and E2.

profile with reincarnation as the failure model: node crashes with volatile-state loss — the crashed node reincarnates rather than crash-recovers — plus partitions, with a Maelstrom adapter fronting the lock service as a compare-and-set register and competing lock clients at the lease cadence. The design runs the fault profile at three nodes, then at five, `xxxx` runs per cluster size. Three checks decide each run: Maelstrom’s linearizability check passes over the register — no committed operation is lost, and no node rejoins under its old identity to overwrite a newer decision; every killed node reincarnates and rejoins serving, with no run ending in a permanently lost node; and no read ever reports two holders, expiry being judged only in the leader’s clock. All figures `xxxx`. A passing campaign supports the case that reincarnation eliminates the FAST’18 weakness: consistency under crash with successful rejoin, by construction rather than by recovery.

D. E4: cloud repetition

E1 and E2 are repeated on commodity cloud virtual machines, the six-node cluster laid out over three regions, two machines each — the correlated-infrastructure setting that weighted configurations address. Variables, procedure, metrics, and sample counts are E1’s and E2’s unchanged; only the environment differs. The planned report records `xxxx` kills per configuration and zone-crossing medians of `xxxx` ms (E1) and `xxxx` ms (E2), with `xxxx` availability incidents. Fig. 9 shows the designed chart: experiment on the horizontal axis (E1 rejoin, E2 flush delta), latency on the vertical axis, one series per deployment (local cluster, cloud machines). No data points exist yet; the chart is watermarked “dummy”.

TABLE IV
ABDICATION FAILOVER, THREE CYCLES (MS). *drive* IS THE OPERATOR VERB’S REPLY LATENCY; *failover* IS ABDICATION-DRIVE TO SUCCESSOR VIEW’S FIRST COMMIT; *settle* IS DRIVE TO ALL THREE NODES REPORTING NORMAL IN THE NEW VIEW.

Cycle	Abdicating leader	drive	failover	settle
1	n2	7	30	145
2	n3	8	27	156
3	n1	8	26	148

WARNING: THIS WORK HAS NOT BEEN DONE. The experiments below are designs; all numbers and charts are placeholders (marked “dummy”).

X. MAINTENANCE OPERATIONS: FIRST MEASUREMENTS

The experiments of Section IX are the campaign’s designs. This section reports the first measured evidence, from a preliminary maintenance-operations run over the `lunet-locks` demonstration: a three-node cluster on one host, deliberately configured for calm — a 5 ms heartbeat against a 3,000 ms election timeout (sloppy detector 3,000–5,000 ms), so jitter never fires a suspicion and the baseline churn of the tight timeout cells is absent. Continuous lease traffic runs throughout at roughly 17 operations per second against all three nodes (one `lease-load` client per node). Every measurement below comes from the harness’s own anchors, the node logs, and the flight-recorder tapes; the complete run artefacts (logs, superblock images, flight tapes, and a dated `snapshot_run` archive) are retained.

A. S2: abdication failover is detection-independent

The run’s central number is the leader-abdication failover time. Three cycles were driven: at each cycle the serving leader was asked to abdicate through the operator verb, the abdication committed (`accepted: true`), and the cluster installed the next view with a new leader — the rotation `n2 → n3 → n1 → n2` covering all three nodes. The measurement spans the abdication drive’s reply to the installation of the successor view’s first commit. Because the abdication itself proposes the view change, no failure detection participates: the number is bounded by the proposal round-trip, not by any timeout.

Table IV gives the cycles. Failover is 26–30 ms against an election-timeout floor of 3,000 ms: two orders of magnitude below the timeout-driven takeover the same cluster would otherwise pay, and independent of the detector by construction. Under load the gap is invisible to clients: across the 24.7 s window containing all three rotations, the longest gap between consecutive client operations at any node was 171 ms — 888 operations issued across the three nodes — at a mean inter-operation interval of about 56 ms, with no client error. The abdicating leader never leaves the cluster: each cycle’s states show the old leader `normal` in the successor view within the settle time (150–181 ms), and leadership had visited every node after three cycles.

B. S1: security-patch rotation through non-leaders

The routine-maintenance scenario is a controlled stop/start of a non-leader: `SIGTERM` (the node's clean-stop path drains and flushes, then exits; the boot gate records the controlled halt), an immediate restart on the same boot line, and rejoin. Two rotations were measured: `n2` stopped in 50 ms and rejoined `normal` in 14,988 ms (first client-visible gap at the other nodes 84 ms); `n3` stopped in 50 ms and rejoined in 19,461 ms. Client serving continued through the first rotation. The rejoin latency here is dominated not by the stop — 50 ms, the cost of the two marker-write rounds — but by the catch-up: the restarted node returns at the boot view and must fold the eras and views the live cluster produced in its absence (39 and 91 view installations respectively), which the current harness walks at its own cadence. The stop/start number itself is the maintenance claim; the catch-up tail is the known cost of churn accumulated during absence, and the Section IX campaign's E1 is designed to measure precisely it at sample counts.

C. Baseline: the timeout-driven takeover

The contrast case is measured twice. On the same calm configuration, the run's S3 `SIGTERM`d the leader: the takeover — successor elected and serving — completed in 20,370 ms (the sloppy detector's window plus the catch-up fold), against S2's 26–30 ms on identical hardware and knobs: three orders of magnitude apart. On the tight timeout cell (heartbeat 5 ms, election 1,000 ms; the softball campaign's attempt-3 gate) the same `SIGTERM` elected a successor correctly but the restarted leader's rejoin took approximately 78 s — the boot-fenced node chasing a churning view space, the failure class Section IX's E1 analyses. The remaining ladder scenarios completed: the full-cluster reboot (all down 30 s, all up) served its first client operation 2,596 ms after restart with all voters `normal` at 2,578 ms, and the controlled reconfiguration swap (decrement, leave, join, increment) completed in 23,542 ms with the cluster serving throughout. The crash-restart scenario is recorded blocked-with-evidence: a crashed node's rejoin at a deep view sat fenced for the full 150 s budget (the era-fold/boot-fence defect family, a liveness defect of the harness's rejoin walk, under active debugging); the flight tapes show no committed state lost and no superseded identity voting again. The full detail, with anchors and provenance, is Supplement S1 and the lab books of `papers/results/`.

D. Status

The evidence state is: abdication failover measured at 26–30 ms, detection-independent, client-invisible (Section X-A); non-leader stop/start measured at 50 ms stop plus a churn-dominated rejoin tail (Section X-B); the timeout-driven baseline measured as the softball campaign's 78 s rejoin gate failure (Section X-C). Nothing here yet measures the E1–E4 campaigns at their designed sample counts; the placeholders of Section IX stand until those runs.

XI. REPRODUCIBILITY AND RESEARCH METHODOLOGY

A. Executable evidence

Each positive rung contains the entire module text, a build invocation, and axiom reports for its stated results. `showboat verify` reruns the commands and compares their captured output [11]. All eight inherited documents pass this check. The eighth contains the expected compiler failure of a historical Leanstrat draft, so its successful replay does not establish the missing weighted theorem. Rung 9 supplies that theorem.

The checked additions are numbered 9 (general weights), 10 (operational acceptor), 11 (suffix promises), 12 (fault controls), 13 (whole-log selection), 14 (normal-log provenance), 15 (fenced histories), and 16 (shared multi-view preservation), 17 (conditional recovery fencing), 18 (crash-vector collection), 19 (acquisition ordering), and 20 (operational acquisition certificates). The classic crash-recover attempt's delayed-fence counterexample rung was deleted with that attempt, and the ladder's recovery rungs are numbered 17–20 after renumbering, leaving number 21 retired. The ladder continues with rung 22 (the reincarnation specification: the two-era forced weight sequence and the four-superblock identity state machine, with its general theorems recorded as obligations) and rung 23 (the casting vote instantiated on the reincarnation eras, with the degenerate three-node negative). The original report is retained as an audit input. Its statement that every rung includes a successful build and axiom report does not hold for rung 8.

The independent baseline audit ran the unmodified library build and every original transcript, inspected theorem types and axiom reports, and preserved input hashes. Subsequent changes were checked in small increments. The library build and transcript checks complement the Rust format, lint, test and documentation gates. Passing the Rust suite does not establish a formal refinement to Lean. A vendor-directory match in an inherited repository text gate is documented rather than suppressed by modifying unrelated code.

An independent TLC 2.19 breadth-first run of `VrrCoreEras.cfg` completed in 137.67 seconds, generating 2,743,933 states and finding 837,204 distinct states, with no error reported and an empty remaining queue. Its scope is the three-node `inc3` scenario, one command value, maximum log length three, two eras, and view index zero. `MaxEpoch=0` disables crashes; this run supplies no crash-recovery coverage. TLC reported a fingerprint collision probability estimate of 2.2×10^{-7} from the observed fingerprints. This finite exploration is separate from kernel-checked unbounded theorems. The `VrrCoreErasM1.cfg` transfer mutation produced the expected `FrontiersOrdered` violation in 1.2 seconds. Source and configuration hashes and full logs are retained for both runs.

A single executable document, `REPRODUCE.md`, packages the whole chain: the exact tool versions (Lean 4.33.1, Lake 5.0.0, Showboat 0.6.1, TLC 2.19 from the v1.7.4 release of `tla2tools.jar` pinned by SHA-256, Tectonic 0.17.0 and the Rust 1.96.0 toolchain), the library build, the axiom audit, every rung replay, the mutation controls, the two TLC runs, the Rust gates, and verification of the published paper. `showboat verify REPRODUCE.md` reruns all of it and diffs the cap-

tured output. The paper is published by `paper/build.sh` as `papers/<id>.pdf`, where the id is the build date plus the short HEAD sha: an immutable version of record, stamped into every page footer and asserted with `pdftotext` before publication. `REPRODUCE.md` therefore verifies the published PDF for the current id by digest and footer instead of rebuilding it. The build fixes `SOURCE_DATE_EPOCH`, so a given Tectonic release and package bundle typeset a stable layout, but publication is by id, not by byte-identical rebuild. The mutated-configuration TLC run is replayed with one worker, because a multi-worker search stops at a nondeterministic point after the first violation; only the violated invariant is compared.

B. Bounded multi-model proof assistance

Leanstral is a Lean-oriented model available through Mistral’s tooling. Its release article reports benchmarks and case studies [12]; those results do not establish cost effectiveness for this core-only development. Here the model is a source of candidate proof text. The local Lean kernel, the theorem type, and an explicit axiom policy determine acceptance.

The inherited record reports a failed attempt on the general weighted lemma, including a large Lean process. We preserved that record and replayed the draft’s compiler failure. We did not independently reproduce its historical peak memory or elapsed time.

The new experiment isolated a single list-induction obligation with unchanged definitions and statement. The first Vibe run had a 12-turn, 28,000-token, 180-second and 2-GiB sampled process-tree memory limit, with a configured USD 2 cap. It stopped after 8.19 seconds when scaffold usage exceeded the token limit, before a proof edit. The second used 6 turns, 150,000 tokens, 120 seconds and a configured USD 0.50 cap. It stopped after 76.39 seconds on the cumulative-token limit, leaving the original proof hole. Peak sampled resident memory was about 205 MiB in each run. Neither cap is a measurement of actual billing, and sampling is not an operating-system memory guarantee. Two direct-API routing probes also failed. The weighted theorem was then completed and compiled locally.

No accepted Leanstral-generated proof is attributed to these runs. Their methodological result is that scaffold overhead dominated a tiny proof target; repeating the same configuration was not justified. Future comparisons should record exact prompts, tool context, candidate edits, compiler iterations, token usage where available, elapsed time, and invoiced cost. A claim of multi-model efficiency needs comparable successful obligations and a baseline workflow. This experiment supplies neither a measured dollar saving nor a general negative judgment about the model.

The laboratory book distinguishes recorded authorship of earlier work from independently checked artifacts. This permits credit to prior model assistance without making model identity part of the trusted proof base.

XII. RELATED WORK

Viewstamped Replication was introduced by Oki and Liskov [1]: a primary-copy method in which computations run at a

primary that notifies its backups, view changes reorganize the replicas when a failure occurs, and a viewstamp — a timestamp concatenated with a view identifier — detects lost information cheaply. VRR-2012 [2] is the updated protocol this paper treats as the reference model: state machine replication for services such as a lock manager, diskless operation with replicated state providing persistence, two-round view changes selecting the log with the greatest last-normal view, a fresh-nonce recovery exchange, and reconfiguration through epochs. VR was developed independently of the Paxos line, and differs from a one-shot consensus protocol in that it specifies client handling, recovery, and reconfiguration as part of one protocol.

Turner’s paper [3] is the construction retold here: it keeps the consensus pipeline unbounded even while the cluster reconfigures, subsuming the bounded-pipeline regimes and the restricted-reconfiguration regimes that preceded it, and generalizing simple majorities to integer-weighted majorities that tolerate correlated infrastructure failures during maintenance. His account surveys those prior regimes by name — Lamport’s bounded pipeline, and the variants the literature calls Dynamic Paxos, Cheap Paxos, Stoppable Paxos, Vertical Paxos, Egalitarian Paxos, Raft, and Zab’s limited-pipeline reconfiguration — and we retain his terms only in that contrast. His appendices are informal versions of Isabelle/HOL proof efforts; this paper replaces that evidence basis with a Lean development whose positive results are kernel-checked and replayable, adds the negative controls, and extends the construction’s boundary to the VRR recovery and reincarnation obligations.

Committed-data loss on crash recovery was documented at the systems level by Alagappan et al. [4] (their Section 2.3), whose protocol-aware recovery makes the storage layer enforce the protocol’s own invariants, and formalized for diskless crash recovery by Michael et al. [5], [6], whose crash-vector collection and acquisition ordering are checked as ladder rungs here. uVRR differs from both in that it removes the class by construction: no recovered-but-amnesiac identity ever rejoins, so the storage layer is never asked to repair one. The voting-weights rules the reincarnation leader drives are published in [9], and the economics of trading fsyncs for network round trips are argued in [8]. The motivating proposal is [7].

XIII. CONCLUSION AND OPEN OBLIGATIONS

The shared crash-free model now preserves quorum-voted prefixes across view changes, and the ladder stands at its twenty-two rungs (numbers 1 through 23, with 21 retired) with every positive result kernel-checked and replayable as of this commit. The next integration theorem must relate the activation, message-history, and local-state abstractions to the concrete VRR protocol. The Crash-Stop-Self-Evict reincarnation path (the four-superblock durable identity, the reincarnation message, and the forced weight sequence) must be proved against the fence and acquisition machinery checked here; its general theorems are recorded as obligations with the reincarnation rung. Finally, reconfiguration must preserve these facts for an arbitrary sequence of membership changes and state transfers. These are concrete proof obligations, not consequences of quorum intersection alone.

The term *non-stop* also needs an operational definition. A useful conditional progress statement would quantify over an available casting-vote schedule and fair delivery while allowing arbitrary pipeline length. It would state which classes of requests keep completing while the new phase-I exchange is pending. It would not promise service after loss of all necessary quorums. Similarly, no fixed failover time follows in an asynchronous model allowing unbounded message delay. A single-digit-millisecond result requires specified failure detection, network and workload conditions, an implementation, and measured latency distributions — which is precisely what the designed experiments of Section IX do not yet supply.

The present results establish exact component theorems with executable evidence and targeted counterexamples. The demonstration implements the diskless core for advisory leases and is under active hardening. Completing the operational composition will determine the scope of the final uVRR correctness claim; performance and novelty claims will be evaluated against that completed specification and its experimental evidence.

APPENDIX REPRODUCTION COMMANDS

From the repository root, with the tools listed in `formal/uvrr-lean/REPRODUCE.md` installed at the recorded versions:

```
export PATH="$HOME/.elan/bin:$PATH"
cd formal/uvrr-lean
showboat verify REPRODUCE.md
```

That document lists every command with its captured output, and `showboat extract REPRODUCE.md` emits a shell script that recreates it. The document runs the pinned Lean toolchain declared in `lean-toolchain`. Showboat transcripts compare command output, not only exit status, so a fresh library build is recorded independently of transcript replay. Historical rejected drafts are quarantined outside the compiled library.

REFERENCES

- [1] B. M. Oki and B. H. Liskov, “Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems,” in *Proc. Seventh Annual ACM Symp. Principles of Distributed Computing (PODC 1988)*, Edmonton, Alberta, Canada, Aug. 1988, pp. 8–17.
- [2] B. Liskov and J. Cowling, “Viewstamped Replication Revisited,” MIT-CSAIL-TR-2012-021, MIT Computer Science and Artificial Intelligence Laboratory, Cambridge, MA, 23 July 2012. <https://hdl.handle.net/1721.1/71763>.
- [3] D. C. Turner, “Unbounded Pipelining in Dynamically Reconfigurable Paxos Clusters,” revision 1A9DBA37, 14 August 2017. <https://tessanddave.com/paxos-reconf-latest.pdf>.
- [4] R. Alagappan, A. Ganesan, E. Lee, A. Albarghouthi, V. Chidambaram, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Protocol-Aware Recovery for Consensus-Based Storage,” in *Proc. 16th USENIX Conf. File and Storage Technologies (FAST 18)*, Oakland, CA, Feb. 2018, pp. 15–32. Best Paper Award.
- [5] E. Michael, D. R. K. Ports, N. K. Sharma, and A. Szekeres, “Providing Stable Storage for the Diskless Crash-Recovery Failure Model,” UW-CSE-16-08-02, 25 August 2016. <https://homes.cs.washington.edu/~drkp/papers/diskless-tr16.pdf>.
- [6] E. Michael, D. R. K. Ports, N. K. Sharma, and A. Szekeres, “Recovering Shared Objects Without Stable Storage [Extended Version],” UW-CSE-17-08-01, 5 August 2017; extended version of DISC 2017 paper. <https://drkp.net/papers/recovery-tr17.pdf>.

- [7] simbo1905, “Unbounded Viewstamped Replication Revisited?” 12 August 2026. <https://simbo1905.wordpress.com/2026/08/12/viewstamped-replication-revisited/>.
- [8] simbo1905, “The network is faster than the disk,” 12 April 2024, accessed 7 September 2026. <https://simbo1905.wordpress.com/2024/04/12/the-network-is-faster-than-the-disk/>.
- [9] simbo1905, “Paxos Voting Weights,” 16 March 2017, accessed 7 September 2026. <https://simbo1905.wordpress.com/2017/03/16/paxos-voting-weights/>.
- [10] simbo1905, blog posts on two-era recovery and leader overlap, Paxos category archive, accessed 8 September 2026. <https://simbo1905.wordpress.com/category/paxos/page/2/>.
- [11] S. Willison, “Showboat: Create executable demo documents,” software, version 0.6.1 used for the inherited evidence. <https://github.com/simonw/showboat>.
- [12] Mistral AI, “Leanstral: Open-Source foundation for trustworthy vibecoding,” 16 March 2026, accessed 6 September 2026. <https://mistral.ai/news/leanstral/>.